

Docker Fundamentals

1. Introduction to Docker

Docker is a containerization platform that packages applications and dependencies into isolated, lightweight containers. It enables consistent deployment across different environments, from development to production, solving the 'works on my machine' problem.

2. Core Concepts

Container: A lightweight, standalone, executable package containing code, runtime, system tools, and libraries.

Image: A blueprint for creating containers. It's a read-only template that includes everything needed to run an application.

Dockerfile: A text file containing instructions to build a Docker image.

Docker Hub: A cloud-based registry for storing and sharing Docker images.

3. Dockerfile Instructions

FROM	Sets the base image
RUN	Executes commands in the image
COPY/ADD	Adds files to the image
EXPOSE	Documents the port the container listens on
ENV	Sets environment variables
CMD	Specifies default command to run
ENTRYPOINT	Configures container as executable

4. Docker Commands

docker run	Create and run a container
docker build	Build an image from Dockerfile
docker pull	Download an image
docker push	Upload an image to registry
docker ps	List running containers
docker stop	Stop a running container
docker rm	Remove a container
docker logs	View container logs

5. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications using a YAML file. It simplifies deployment of complex applications with multiple services, networks, and volumes.

6. Best Practices

- Use official base images from Docker Hub
- Keep images small and lightweight
- Use .dockerignore to exclude unnecessary files
- Minimize layers in Dockerfile
- Tag images appropriately
- Implement health checks